

## A Complete Machine Learning Pipeline — Explained

---

*Every section of the code — what it does, how it works, and why it matters*  
*Includes: Data Loading · Preprocessing · Feature Engineering · Model Training ·*  
*Hyperparameter Tuning · Evaluation · Visualisation · Comparison*

pandas

numpy

matplotlib

seaborn

TABLE OF CONTENTS

|    |                                             |       |
|----|---------------------------------------------|-------|
| 01 | Project Overview & Pipeline Architecture    | ..... |
| 02 | Configuration Block                         | ..... |
| 03 | Section 1 — Data Loading                    | ..... |
| 04 | Section 2 — Exploratory Data Analysis (EDA) | ..... |
| 05 | Section 3 — Preprocessing Pipeline          | ..... |
| 06 | Section 4 — Hyperparameter Tuning           | ..... |
| 07 | Section 5 — Model Training                  | ..... |
| 08 | Section 6 — Model Evaluation                | ..... |
| 09 | Section 7 — Visualisations                  | ..... |
| 10 | Section 8 — Final Comparison & Summary      | ..... |
| 11 | ML Concepts Deep Dive                       | ..... |
| 12 | Quick Reference Card                        | ..... |

## What Is This Script?

This Python script implements a complete, production-quality Machine Learning pipeline for binary or multi-class classification problems. It trains two models — a Decision Tree and a Random Forest — on any Kaggle CSV dataset, performs full evaluation, and produces comparison visualisations.

### High-Level Pipeline

| Step | Function / Class      | Purpose                                  |
|------|-----------------------|------------------------------------------|
| 1    | load_data()           | Read CSV, optimise memory                |
| 2    | eda()                 | Print dataset statistics                 |
| 3    | preprocess()          | Clean, encode, split, scale              |
| 4    | tune_decision_tree()  | GridSearchCV for best DT params          |
| 5    | tune_random_forest()  | RandomizedSearchCV for best RF params    |
| 6    | evaluate_model()      | Accuracy, F1, ROC-AUC, CV                |
| 7    | plot_*                | Confusion matrix, ROC, importances, tree |
| 8    | print_final_summary() | Side-by-side winner declaration          |

## What It Does

The configuration block at the top of the script is the only part you ever need to edit to run on a different dataset. It contains three variables:

```
CSV_PATH = "titanic.csv"
```

→ Path to your Kaggle CSV file

```
TARGET_COL = "Survived"
```

→ Name of the column to predict

```
DROP_COLS = ["PassengerId", "Name" ...]
```

→ Columns to remove before training

## Why Separate Configuration?

Keeping all dataset-specific settings in one place (the top of the file) follows the Single Responsibility Principle. You never need to hunt through function bodies to adapt the script — change three lines, run the script, done.

```
RANDOM_STATE = 42
```

Every random operation in the script — train/test split, GridSearchCV shuffles, forest bootstrap sampling — receives this seed. This guarantees that anyone running the same code on the same data gets identical results (reproducibility).

## Function: load\_data(path)

### What Happens

- `pd.read_csv(path, low_memory=False)` — reads the entire CSV into a pandas DataFrame. *low\_memory=False* forces pandas to scan all values before assigning dtypes, preventing mixed-type column surprises.
- `pd.to_numeric(..., downcast='integer')` — converts int64 columns to the smallest integer type that fits (int8, int16, int32). On a 500k-row dataset this can cut memory by 50-75%.
- `pd.to_numeric(..., downcast='float')` — same idea for float64 → float32.
- `df.shape` — prints (rows, columns) so you know what you loaded.
- `df[TARGET_COL].value_counts()` — prints class distribution to spot class imbalance immediately.

### Memory Optimisation Deep-Dive

Large Kaggle datasets (millions of rows) can easily exceed available RAM if loaded naively. By downcasting numeric columns we exploit the fact that most integer features (age, count, year) fit comfortably in int16 rather than int64. Float features (price, ratio) rarely need more than float32 precision for ML. This is a free performance win applied before any processing begins.

| dtype   | Bytes / value | Range                        |
|---------|---------------|------------------------------|
| int8    | 1             | -128 to 127                  |
| int16   | 2             | -32,768 to 32,767            |
| int32   | 4             | -2.1B to 2.1B                |
| int64   | 8             | very large (default)         |
| float32 | 4             | ~7 decimal digits            |
| float64 | 8             | ~15 decimal digits (default) |

## Function: `eda(df)`

EDA (Exploratory Data Analysis) is the discovery phase — understanding the data's shape, quality, and distribution before touching any model.

### What Each Line Does

| Code                                   | What You Learn                                    |
|----------------------------------------|---------------------------------------------------|
| <code>df.info()</code>                 | Column names, dtypes, non-null counts             |
| <code>df.isnull().mean() * 100</code>  | % missing per column → guides imputation strategy |
| <code>df.describe().T</code>           | Mean, std, min, quartiles, max per numeric column |
| <code>df[TARGET].value_counts()</code> | Class distribution → detect imbalance             |

*Key EDA Insight: If any column has >40% missing values, consider dropping it instead of imputing. Imputing half a column with the median introduces artificial noise. The EDA output shows this immediately.*

## Function: preprocess(df)

Preprocessing transforms raw, messy CSV data into clean numeric arrays that ML algorithms can consume. It is the most critical step — garbage in, garbage out.

### Step-by-Step Breakdown

#### Step 4a — Drop Irrelevant Columns

Columns like PassengerId, Name, and Ticket are identifiers — they carry no predictive signal but would confuse the model. We drop them before anything else.

```
df.drop(columns=existing_drops)
```

#### Step 4b — Separate Features & Target

X holds all input features; y holds the labels (what we want to predict). Every operation from here on treats them separately to prevent data leakage.

```
X = df.drop(columns=[TARGET_COL]) y = df[TARGET_COL].copy()
```

#### Step 4c — Identify Column Types

We automatically detect numeric vs. categorical columns using pandas dtype introspection. This makes the pipeline work on any dataset without manual type annotation.

```
num_cols = X.select_dtypes(include=["number"]).columns.tolist() cat_cols =  
X.select_dtypes(include=["object", "category"]).columns.tolist()
```

#### Step 4d — Impute Missing Values

Missing values crash most ML algorithms. We use sklearn's SimpleImputer: • Numeric columns → median (robust to outliers unlike mean) • Categorical columns → most\_frequent (most common category) fit\_transform() learns the fill values from the training data only — this prevents leakage from the test set.

```
SimpleImputer(strategy="median") SimpleImputer(strategy="most_frequent")
```

#### Step 4e — Encode Categorical Features

Machine learning models require numbers. LabelEncoder maps each unique string value to an integer: Male → 0, Female → 1 S → 0, C → 1, Q → 2 For high-cardinality categories or nominal data without order, One-Hot Encoding (pd.get\_dummies) is preferred — you can swap it in here.

```
le = LabelEncoder() X[col] = le.fit_transform(X[col].astype(str))
```

#### Step 4f — Encode Target

If the target column contains strings (e.g. 'yes'/'no', 'cat'/'dog'), we encode it to integers (0, 1). If it's already numeric, this step is a no-op.

```
if y.dtype == object: y = le.fit_transform(y.astype(str))
```

#### Step 4g — Train / Test Split

We split 80% of the data for training and 20% for testing. `stratify=y` ensures both splits have the same class proportions as the full dataset — essential for imbalanced classes.

```
train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)
```

#### Step 4h — Feature Scaling

`StandardScaler` transforms each feature to have `mean=0`, `std=1`. Decision Trees and Random Forests are scale-invariant (they use thresholds, not distances), so scaling doesn't improve their accuracy. However it's included for completeness — if you add Logistic Regression or SVM later, the scaled arrays are ready.

```
scaler = StandardScaler() X_train_sc = scaler.fit_transform(X_train)
```



## What Is a Hyperparameter?

A hyperparameter is a setting you choose before training — unlike model parameters (weights, thresholds) which are learned from data. Examples: `max_depth`, `n_estimators`, `min_samples_leaf`.

## Decision Tree — GridSearchCV

GridSearchCV exhaustively tries every combination of hyperparameters in a grid. For each combination it runs 5-fold cross-validation and picks the combination with the highest mean F1 score.

| Hyperparameter                 | Values Tried      | What It Controls                                                        |
|--------------------------------|-------------------|-------------------------------------------------------------------------|
| <code>max_depth</code>         | 3, 5, 7, 10, None | How deep the tree can grow. Deeper = more complex = risk of overfitting |
| <code>min_samples_split</code> | 2, 5, 10          | Min samples needed to split a node. Higher = simpler tree               |
| <code>min_samples_leaf</code>  | 1, 2, 4           | Min samples required at a leaf. Higher = smoother decision boundary     |
| <code>criterion</code>         | gini, entropy     | Impurity measure: Gini index or Information Gain (entropy)              |

## Random Forest — RandomizedSearchCV

Random Forest has a much larger parameter space. GridSearchCV would take hours. RandomizedSearchCV randomly samples 30 combinations from the distribution and cross-validates them — finding near-optimal settings in a fraction of the time (Bergstra & Bengio, 2012).

| Hyperparameter                 | Values             | What It Controls                                    |
|--------------------------------|--------------------|-----------------------------------------------------|
| <code>n_estimators</code>      | 100, 200, 300, 500 | Number of trees. More = better but slower           |
| <code>max_depth</code>         | 5, 10, 15, None    | Max depth of each tree                              |
| <code>min_samples_split</code> | 2, 5, 10           | Min samples to split a node                         |
| <code>min_samples_leaf</code>  | 1, 2, 4            | Min samples at leaf                                 |
| <code>max_features</code>      | sqrt, log2, None   | Features considered at each split. sqrt is standard |
| <code>bootstrap</code>         | True, False        | Whether to use bootstrap sampling                   |

*Stratified K-Fold Cross-Validation: Both search methods use `StratifiedKFold(n_splits=5)`. This divides the training data into 5 equal folds, trains on 4, validates on 1, and rotates — giving 5 validation scores. Stratified means each fold maintains the original class ratio.*

## Decision Tree — How It Learns

A Decision Tree recursively partitions the feature space using a series of if-then rules. At each node, it searches for the single feature and threshold that best separates the classes — measured by the Gini impurity or Information Gain (entropy).

### Gini Impurity

$Gini = 1 - \sum(p_i^2)$  where  $p_i$  is the proportion of class  $i$  at a node.

A node with all samples from one class has  $Gini = 0$  (pure). A node split 50/50 has  $Gini = 0.5$  (worst). The algorithm picks the split that minimises the weighted Gini of the two child nodes.

### Entropy / Information Gain

$Entropy = -\sum(p_i * \log_2(p_i))$  Information Gain = Parent entropy - weighted child entropy

Both criteria find the same splits in practice. Gini is slightly faster to compute; entropy can produce more balanced trees.

## Random Forest — How It Learns

A Random Forest is an ensemble of Decision Trees. Three sources of randomness make each tree unique and decorrelated from the others:

### 1. Bootstrap Sampling

Each tree is trained on a random sample (with replacement) of the training data — roughly 63% unique rows. The 37% not selected is called the Out-of-Bag (OOB) set and can be used for internal validation.

### 2. Feature Randomness

At each split, only a random subset of features is considered ( $\text{max\_features} = \text{'sqrt'} \rightarrow \sqrt{n\_features}$ ). This prevents strong features from dominating every tree, forcing trees to find different patterns.

### 3. Majority Vote

For prediction, each of the  $N$  trees casts a vote. The class with the most votes wins. This averaging dramatically reduces variance compared to a single tree.

*Why Does Averaging Reduce Variance? If each tree has error  $e$  and errors are uncorrelated, averaging  $N$  trees reduces variance by  $\sim 1/N$ . Bootstrap + feature randomness ensures the trees are as uncorrelated as possible.*

## Function: `evaluate_model()`

A single accuracy number is not enough to trust a model. `evaluate_model()` computes a full suite of metrics, each answering a different question.

| Metric     | Formula                     | What It Answers                                                   |
|------------|-----------------------------|-------------------------------------------------------------------|
| Accuracy   | Correct / Total             | Overall correctness (misleading for imbalanced classes)           |
| Precision  | $TP / (TP + FP)$            | Of all predicted positives, how many were right?                  |
| Recall     | $TP / (TP + FN)$            | Of all actual positives, how many did we catch?                   |
| F1-Score   | $2 * (P * R) / (P + R)$     | Harmonic mean of Precision & Recall — balanced metric             |
| ROC-AUC    | Area under the ROC curve    | Probability model ranks a random positive above a random negative |
| CV F1 Mean | Mean of 5-fold F1 scores    | How consistent is the model across different data subsets?        |
| CV F1 Std  | Std dev of 5-fold F1 scores | Low std = stable model; high std = may be overfitting             |

## Confusion Matrix Explained

The confusion matrix shows every type of prediction error in a single table.

|                  | Predicted: Negative                                    | Predicted: Positive                                    |
|------------------|--------------------------------------------------------|--------------------------------------------------------|
| Actual: Negative | TN (True Negative)<br>Correctly said No                | FP (False Positive)<br>Wrongly said Yes — Type I Error |
| Actual: Positive | FN (False Negative)<br>Wrongly said No — Type II Error | TP (True Positive)<br>Correctly said Yes               |

## Overfitting Check

The script computes both train accuracy and test accuracy. The difference (generalisation gap) tells you about over/underfitting:

```
train_acc = accuracy_score(y_train, model.predict(X_train)) gap = train_acc - test_acc
```

| Gap      | Interpretation                              | Fix                                                                    |
|----------|---------------------------------------------|------------------------------------------------------------------------|
| > 0.10   | Overfitting — model memorised training data | Increase <code>min_samples_leaf</code> , reduce <code>max_depth</code> |
| ~0.00    | Good generalisation                         | No action needed                                                       |
| Both low | Underfitting — model too simple             | Deeper tree, more estimators, better features                          |

## Every Plot — What It Shows & Why

### `plot_confusion_matrix()`

*File: confusion\_decision\_tree.png / confusion\_random\_forest.png*

Uses sklearn's `ConfusionMatrixDisplay.from_estimator()` to draw a colour-coded grid of TP, FP, TN, FN counts. Blue intensity = count. One plot per model lets you see where each model makes different types of errors.

### `plot_roc_curves()`

*File: roc\_curves.png*

Plots the Receiver Operating Characteristic curve for both models overlaid. The x-axis is False Positive Rate; the y-axis is True Positive Rate. A perfect model hugs the top-left corner (AUC=1.0). A random model is the diagonal (AUC=0.5). Only available for binary classification.

### `plot_feature_importance()`

*File: feat\_importance\_decision\_tree.png / feat\_importance\_random\_forest.png*

Reads `model.feature_importances_` — the fraction of total impurity reduction attributable to each feature across all splits. Displayed as a horizontal bar chart sorted by importance. Tip: RF importances are more reliable than DT importances because they are averaged over many bootstrap samples.

### `plot_decision_tree()`

*File: decision\_tree\_structure.png*

Uses sklearn's `plot_tree()` to draw the first 3 levels of the decision tree as a flowchart. Each node shows: feature name, threshold, Gini/entropy value, sample count, and class distribution. Also prints the rules as text via `export_text()`.

### `plot_comparison()`

*File: model\_comparison.png*

Side-by-side grouped bar chart of Accuracy, Precision, Recall, and F1 for both models. Makes the performance difference instantly visible.

### `plot_cv_distributions()`

*File: cv\_distributions.png*

Violin plot of 5-fold cross-validation F1 scores for both models. A narrow violin means the model is consistent; a wide or bimodal violin signals instability. Dots inside show individual fold scores.

## Function: `print_final_summary()`

This function prints a formatted table comparing all metrics side-by-side, declares a winner (by weighted F1), and provides a list of key observations and recommendations. Here is what each observation means:

### Observation 1 — RF generalises better

Averaging  $N$  decorrelated trees reduces prediction variance without increasing bias. A single Decision Tree is one hypothesis; a Random Forest is a committee.

### Observation 2 — DT is interpretable

You can follow the exact path from root to leaf for any prediction. In regulated industries (finance, healthcare) this explainability is legally required.

### Observation 3 — Lower CV std for RF

If the CV standard deviation for Random Forest is smaller, it means the model performs similarly across different random subsets — a sign of robustness.

### Observation 4 — DT overfitting

An unconstrained Decision Tree will grow until it memorises every training sample (100% train accuracy, poor test accuracy). `GridSearchCV` limits `max_depth` and `min_samples_leaf` to control this.

### Observation 5 — Feature importances

Both models expose `feature_importances_` but RF importances are averaged over many trees fitted on bootstrap samples — statistically more stable.

## Bias-Variance Trade-Off

---

Bias is error from wrong assumptions (underfitting). Variance is error from sensitivity to training data (overfitting). Decision Trees have low bias but high variance. Random Forest reduces variance by averaging — it's the central reason ensembles outperform single models.

## Bootstrap Sampling (Bagging)

---

Each tree in the forest is trained on a random sample drawn WITH replacement from the training set. On average, each sample contains ~63.2% unique rows. The remaining ~36.8% (Out-of-Bag samples) can be used to estimate generalisation error without a separate validation set.

## Gini vs Entropy

---

Both measure node impurity. Gini:  $1 - \sum(p_i^2)$ . Entropy:  $-\sum(p_i \log_2(p_i))$ . In practice they produce nearly identical trees. Entropy is slightly more expensive to compute (requires a log). Gini tends to isolate the most frequent class into its own branch; entropy builds more balanced trees.

## Cross-Validation (K-Fold)

---

Instead of a single train/test split (which depends on luck), K-Fold CV divides data into K equal parts, trains on K-1, tests on the remaining 1, and rotates K times. The average score across folds is a much more reliable estimate of real-world performance.

## GridSearchCV vs RandomizedSearchCV

---

GridSearchCV: tries every combination. With 5 params x 3 values each = 243 combinations x 5 folds = 1,215 model fits. Slow but exhaustive. RandomizedSearchCV: randomly samples  $n\_iter$  combinations.  $n\_iter=30$  x 5 folds = 150 fits. Much faster with near-equal quality for large parameter spaces.

## Feature Importances

---

sklearn computes importance as: for each feature, sum the total reduction in impurity (Gini/entropy) it caused across all nodes and all trees, weighted by the number of samples that passed through each node. Result is normalised to sum to 1. High importance = feature is used early (close to root) and/or often.

## ROC Curve & AUC

---

The ROC curve is generated by varying the classification threshold (default 0.5) from 0 to 1. At each threshold we plot (FPR, TPR). AUC is the area under this curve: 1.0 = perfect, 0.5 = random. AUC is threshold-independent — it measures the model's ability to rank positives above negatives regardless of cutoff.

## 12 | QUICK REFERENCE CARD

### Adapting to Any Kaggle Dataset

Only three lines need changing:

```
CSV_PATH = "your_dataset.csv" TARGET_COL = "your_label_column" DROP_COLS = ["id_col",  
"leak_col"]
```

### Common Issues & Fixes

| Issue                        | Cause                       | Fix                                          |
|------------------------------|-----------------------------|----------------------------------------------|
| KeyError: 'Survived'         | Wrong TARGET_COL            | Check df.columns after loading               |
| MemoryError                  | Dataset too large           | Use chunksize in pd.read_csv or sample       |
| All features numeric — error | No categorical columns      | cat_cols will be empty; imputer skipped      |
| ROC-AUC not computed         | Multiclass target           | Extend with roc_auc_score(multi_class='ovr') |
| Overfitting (gap > 0.10)     | Tree too deep               | Reduce max_depth in GridSearchCV grid        |
| Poor accuracy for all models | Underfitting / bad features | Add feature engineering, check TARGET_COL    |
| GridSearchCV very slow       | Large grid                  | Switch to RandomizedSearchCV with n_iter=20  |

### When to Use Which Model

| Situation                                 | Recommended Model                         |
|-------------------------------------------|-------------------------------------------|
| Need to explain decisions to stakeholders | Decision Tree                             |
| Maximum predictive accuracy               | Random Forest                             |
| Very small dataset (< 500 rows)           | Decision Tree (less overfitting risk)     |
| Large dataset (> 100k rows)               | Random Forest (or LightGBM next step)     |
| Real-time prediction (latency matters)    | Decision Tree (single if-else path)       |
| Feature selection / importance ranking    | Random Forest (more reliable importances) |
| Regulated industry (audit trail required) | Decision Tree                             |

End of Guide — Happy Modelling!